Multi-Tenant Companion Execution Blueprint

Scope of This Companion

This document adds three implementation-grade artifacts to the first strategy report:

1. Firestore schema v1 for multi-tenant operations
2. 20-week delivery roadmap with milestones, ownership, and quality gates
3. Migration map from current Zara modules into the new multi-tenant system

Assumptions for delivery model:

- Development is executed in VS Code
- GitHub Copilot acts as implementation developer
- You act as prompt engineer, reviewer, and product decision owner

---

1) Firestore Schema v1 (Tenant, Location, Staff, Booking, Engagement)

1.1 Design Principles

- Every business object is tenant-scoped by default
- Location is explicit where operations depend on physical capacity
- Staff ownership is explicit for schedule and payout logic
- Event logs are append-only where auditing is needed
- Query patterns drive indexes before feature launch

1.2 Core Collections

tenants

Purpose: top-level business entity (one company/salon brand)

Document shape:

```
{
  "name": "Luna Nails Group",
  "slug": "luna-nails",
  "status": "active",
  "ownerUserId": "uid_owner_123",
  "plan": "pro",
  "country": "HR",
  "defaultLanguage": "en",
  "defaultCurrency": "EUR",
  "timezone": "Europe/Zagreb",
  "branding": {
    "logoUrl": "https://...",
    "primary": "#1F4D3A",
    "secondary": "#F2E7D5",
```

```
      "accent": "#C08A45",
      "fontHeading": "Manrope",
      "fontBody": "Inter",
      "radius": 12
    },
    "settings": {
      "bookingLeadHours": 2,
      "bookingMaxDays": 90,
      "cancellationWindowHours": 24,
      "allowGuestBooking": false,
      "requireDeposit": false
    },
    "createdAt": "serverTimestamp",
    "updatedAt": "serverTimestamp"
  }
```

tenantUsers

Purpose: many-to-many mapping between users and tenant roles (extended for subscription tracking)

Document id recommendation: tenantId_userId

```
  {
    "tenantId": "tenant_abc",
    "userId": "uid_456",
    "role": "tenant_admin",
    "status": "active",
    "permissions": ["bookings.read", "bookings.write", "campaigns.write"],
    "locationIds": ["loc_1", "loc_2"],
    "subscription": {
      "tier": "professional",
      "status": "active",
      "billingCycle": "monthly",
      "startDate": "2026-04-01",
      "trialEndsAt": null,
      "nextBillingDate": "2026-05-01",
      "suspendedAt": null,
      "suspensionReason": null
    },
    "createdAt": "serverTimestamp",
    "updatedAt": "serverTimestamp"
  }
```

Roles recommended:

- platform_admin
- tenant_owner
- tenant_admin
- location_manager
- technician

- client

Subscription tiers:

- free_trial (14 days, limited features)
- starter (basic features)
- professional (most features)
- enterprise (all features + custom support)

Subscription statuses:

- trialing (14-day free trial active)
- active (paid subscription active)
- past_due (payment failed, grace period active)
- suspended (no payment, features locked)
- cancelled (user cancelled)

locations

Purpose: physical salon branches under a tenant

```
{
  "tenantId": "tenant_abc",
  "name": "Luna Nails Downtown",
  "code": "DOWNTOWN",
  "status": "active",
  "timezone": "Europe/Zagreb",
  "phone": "+385...",
  "email": "downtown@...",
  "address": {
    "line1": "Ilica 12",
    "city": "Zagreb",
    "country": "HR",
    "postalCode": "10000",
    "lat": 45.81,
    "lng": 15.97
  },
  "operatingHours": {
    "mon": [{ "start": "09:00", "end": "19:00" }],
    "tue": [{ "start": "09:00", "end": "19:00" }]
  },
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

staff

Purpose: technicians and employees

```
{
  "tenantId": "tenant_abc",
  "locationIds": ["loc_1"],
  "userId": "uid_tech_1",
  "displayName": "Mia K.",
  "role": "technician",
  "status": "active",
  "skills": ["gel", "nail_art", "acrylic"],
  "serviceIds": ["svc_gel", "svc_art"],
  "profile": {
    "bio": "Nail art specialist",
    "avatarUrl": "https://..."
  },
  "constraints": {
    "maxConcurrent": 1,
    "breakMinutesDefault": 15
  },
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

staffSchedules

Purpose: source of truth for recurring weekly schedule + exceptions

```
{
  "tenantId": "tenant_abc",
  "staffId": "staff_123",
  "locationId": "loc_1",
  "weekTemplate": {
    "mon": [{ "start": "09:00", "end": "17:00" }],
    "tue": [{ "start": "11:00", "end": "19:00" }]
  },
  "exceptions": [
    { "date": "2026-06-20", "type": "time_off" },
    { "date": "2026-06-21", "type": "custom", "blocks": [{ "start": "12:00",
"end": "16:00" }] }
  ],
  "updatedAt": "serverTimestamp"
}
```

services

Purpose: sellable services (tenant-wide or location-specific)

```
{
  "tenantId": "tenant_abc",
  "locationIds": ["loc_1", "loc_2"],
```

```
  "name": "Gel Manicure",
  "category": "manicure",
  "durationMinutes": 60,
  "bufferMinutes": 10,
  "price": 45,
  "currency": "EUR",
  "active": true,
  "sortOrder": 10,
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

customers

Purpose: tenant-scoped client profile, marketing flags, loyalty state

```
{
  "tenantId": "tenant_abc",
  "userId": "uid_client_1",
  "status": "active",
  "firstName": "Ana",
  "lastName": "M.",
  "email": "ana@...",
  "phone": "+385...",
  "dateOfBirth": "1996-03-12",
  "consent": {
    "emailMarketing": true,
    "pushMarketing": true,
    "smsMarketing": false,
    "termsAcceptedAt": "timestamp",
    "privacyAcceptedAt": "timestamp"
  },
  "preferences": {
    "preferredLocationId": "loc_1",
    "preferredStaffId": "staff_123"
  },
  "loyalty": {
    "points": 820,
    "tier": 2,
    "tierName": "Glow Up",
    "referralCode": "ANAM2026"
  },
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

bookings

Purpose: appointments and lifecycle state machine

```json
{
  "tenantId": "tenant_abc",
  "locationId": "loc_1",
  "staffId": "staff_123",
  "customerId": "cust_456",
  "serviceIds": ["svc_gel", "svc_art"],
  "date": "2026-07-10",
  "startTime": "13:00",
  "endTime": "14:20",
  "status": "confirmed",
  "source": "app_client",
  "notes": "French tip, almond",
  "media": ["https://..."],
  "pricing": {
    "subtotal": 55,
    "discount": 0,
    "final": 55,
    "currency": "EUR"
  },
  "lifecycle": {
    "createdBy": "uid_client_1",
    "confirmedBy": "uid_admin_1",
    "completedAt": null,
    "cancelledAt": null,
    "cancelReason": null
  },
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

Status values recommended:

- pending
- confirmed
- completed
- cancelled
- rejected
- no_show
- reschedule_pending
- reschedule_rejected
- rescheduled

waitlist

Purpose: waiting list requests for date ranges / specific staff / services

```json
{
  "tenantId": "tenant_abc",
  "locationId": "loc_1",
```

```
  "customerId": "cust_456",
  "serviceIds": ["svc_gel"],
  "staffId": null,
  "startDate": "2026-07-01",
  "endDate": "2026-07-15",
  "status": "active",
  "createdAt": "serverTimestamp"
}
```

messages

Purpose: in-app messaging

```
{
  "tenantId": "tenant_abc",
  "senderType": "staff",
  "senderId": "uid_admin_1",
  "receiverType": "customer",
  "receiverId": "cust_456",
  "channel": "in_app",
  "text": "Your booking is confirmed",
  "attachments": [],
  "read": false,
  "createdAt": "serverTimestamp"
}
```

campaigns

Purpose: outbound marketing campaigns

```
{
  "tenantId": "tenant_abc",
  "name": "July Re-Engagement",
  "status": "scheduled",
  "channel": "email",
  "segmentId": "seg_at_risk_30",
  "templateId": "tmpl_001",
  "scheduledAt": "timestamp",
  "metrics": {
    "sent": 0,
    "opened": 0,
    "clicked": 0,
    "converted": 0
  },
  "createdBy": "uid_admin_1",
  "createdAt": "serverTimestamp"
}
```

reviews

Purpose: customer reviews per booking/staff/location

```
{
  "tenantId": "tenant_abc",
  "locationId": "loc_1",
  "staffId": "staff_123",
  "bookingId": "book_789",
  "customerId": "cust_456",
  "rating": 5,
  "comment": "Great service",
  "status": "published",
  "createdAt": "serverTimestamp"
}
```

activities

Purpose: daily/weekly engagement activities and challenges

```
{
  "tenantId": "tenant_abc",
  "type": "challenge",
  "name": "Summer Nail Streak",
  "status": "active",
  "startDate": "2026-07-01",
  "endDate": "2026-07-31",
  "rules": {
    "requiredVisits": 2,
    "rewardPoints": 300
  },
  "audience": "all_customers",
  "createdAt": "serverTimestamp"
}
```

subscriptions (Platform-Level Collection)

Purpose: subscription lifecycle tracking for tenant business accounts

Document id recommendation: auto-generated subscription ID

```
{
  "subscriptionId": "sub_stripe_123abc",
  "tenantId": "tenant_abc",
  "customerId": "cust_stripe_xyz",
  "tier": "professional",
  "status": "active",
  "billingCycle": "monthly",
```

```
  "billingCycleAnnual": false,
  "currentPeriodStart": "2026-04-01",
  "currentPeriodEnd": "2026-05-01",
  "trialEndsAt": null,
  "nextBillingDate": "2026-05-01",
  "cancelledAt": null,
  "cancellationReason": null,
  "payment": {
    "method": "stripe",
    "last4": "4242",
    "brand": "visa"
  },
  "pricing": {
    "monthlyUSD": 29,
    "annualUSD": 290,
    "currency": "USD"
  },
  "failureCount": 0,
  "lastFailureAt": null,
  "gracePeriodEndsAt": null,
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

userTenantAccess (Multi-Salon User Subscriptions)

Purpose: track which users have access to which tenants and keep unread message counts

Document id recommendation: userId_tenantId

```
{
  "userId": "uid_client_1",
  "tenantId": "tenant_abc",
  "accessLevel": "client",
  "subscriptionStatus": "active",
  "subscribedAt": "2026-03-15",
  "unreadMessageCount": 3,
  "lastMessageAt": "2026-04-16T14:32:00Z",
  "lastAccessedAt": "2026-04-16T10:00:00Z",
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

Note on `userTenantAccess`:

- This is a denormalized index to enable fast queries like "list all tenants for user" and "get unread counts per tenant for dashboard"
- Mirrors subset of data from `tenantUsers` and `subscriptions` for performance
- MUST be kept in sync during all subscription/user updates

```
## 1.3 Suggested Indexes (Minimum)
- bookings: tenantId + locationId + date + startTime
- bookings: tenantId + customerId + status + date
- bookings: tenantId + staffId + date + startTime
- customers: tenantId + status + createdAt
- customers: tenantId + loyalty.tier + loyalty.points
- messages: tenantId + receiverId + read + createdAt
- waitlist: tenantId + locationId + status + startDate + endDate
- campaigns: tenantId + status + scheduledAt
- reviews: tenantId + locationId + staffId + createdAt
- subscriptions: tenantId + status
- subscriptions: status + nextBillingDate
- subscriptions: status + gracePeriodEndsAt (for payment failure grace period
queries)
- userTenantAccess: userId + accessLevel
- userTenantAccess: userId + subscriptionStatus + lastAccessedAt (for dashboard
multi-salon listing)
- userTenantAccess: tenantId + subscriptionStatus (for tenant's active user count)

## 1.4 Security Rule Guidance (High-Level)
- Enforce request.auth != null globally except public assets
- Validate tenant access via tenantUsers mapping
- Every read/write must assert resource.data.tenantId in callerTenantScope
- Restrict role-sensitive paths (campaigns, staff, pricing, reporting)
- Denormalize safely but never skip tenantId validation

---

## 2) 20-Week Execution Roadmap (Copilot-Driven Delivery)

## Team Operating Model
- Prompt Engineer (you): writes feature specs, acceptance criteria, and review
prompts
- Copilot (developer): implements code, tests, migrations, docs in VS Code
- Decision cadence: weekly architecture review + daily delivery check-ins

## Definition of Done (all weeks)
- Code merged with tests
- Firestore rules and indexes updated
- Documentation updated
- Manual acceptance checklist passed
- No high-severity regression in smoke tests

## Scope Reality and Phasing Decision
Approved scope now includes: marketplace, salon onboarding, client onboarding
integration, configurable free trial, Stripe Billing + Stripe Connect, and broad
AI feature delivery.

Recommended sequence:
1. Build core platform reliability first (Weeks 1-12)
2. Add monetization and onboarding hardening (Weeks 13-16)
```

```
3. Launch marketplace + AI layer in controlled phases (Weeks 17-20)

This is the lowest-risk path that preserves tenant isolation and release quality.

## Week-by-Week Plan

### Week 1: Foundation and Repo Setup
Objectives:
- New repository and baseline architecture
- Firebase project bootstrap and environments
- CI, lint, test harness, feature flag scaffolding
- Public-first route architecture (Landing before auth)

Deliverables:
- Monorepo or app repo structure finalized
- Base contexts: Auth, Tenant, Theme, Localization
- Firestore schema v1 draft committed
- Public and protected route groups with auth guards
- Landing screen with Login, Create Account, and Discover CTA

Quality gates:
- App boots on web + Android + iOS simulator
- Tenant-aware auth guard works end-to-end

### Week 2: Tenant and Location Core
Objectives:
- Tenant onboarding data model
- Location CRUD and listing
- Role model + tenant user mapping **with subscription metadata** (for future
multi-salon + billing)
- Marketplace-ready discovery entry scaffold
- Build userTenantAccess denormalized index for multi-salon user support

Deliverables:
- tenants, tenantUsers (extended with subscription fields), locations modules
- userTenantAccess index collection and repository
- Admin screens for tenant/location setup
- Role-aware navigation skeleton
- DiscoverBusinesses screen scaffold using mocked data contracts

Quality gates:
- Cross-tenant access denied by security rules
- Subscription fields in tenantUsers properly structured but not enforced yet
(enforcement after Stripe integration)
- Public discover flow works without auth while protected app remains guarded

### Week 3: Service and Staff Modules
Objectives:
- Service catalog per tenant/location
- Staff profiles and service skills mapping

Deliverables:
- services and staff modules
- staffSchedules initial structure
```

```
- Admin CRUD workflows

Quality gates:
- Services and staff fully filtered by tenant/location

### Week 4: Scheduling Engine v1
Objectives:
- Availability computation by staff + location + service duration
- Conflict prevention rules

Deliverables:
- Slot generation utility
- Calendar primitives for admin and client
- Unit tests for slot edge cases

Quality gates:
- No double booking in concurrent tests

### Week 5: Booking Lifecycle v1
Objectives:
- Booking creation + status transitions
- Client and admin booking views

Deliverables:
- bookings collection and state machine
- Confirm/reject/cancel flows
- Reschedule request flow v1

Quality gates:
- Lifecycle transitions only via authorized roles

### Week 5.5: Multi-Salon Home Dashboard and Context Switcher (NEW)
Objectives:
- Build user home screen showing all subscribed salons
- Real-time badge counts for unread messages per salon
- Quick context switching and deep-linking into each salon
- Marketplace discovery integration if user has no subscriptions

Note: This is distinct from the single-salon app context. After authentication,
users land on this dashboard to *choose* which salon to enter. Once inside a salon
context, the app behaves as single-tenant.

Deliverables:
- MultiSalonDashboardScreen component with salon cards
- Unread message count aggregation service across all user's tenants
- Context switcher (select salon → navigate into protected app with tenantId set)
- Salon card UI showing: logo, name, next appointment, unread count, quick actions
(Book, Message, Profile)
- Loyalty points balance per salon (from userTenantAccess denormalized data)
- Integration into auth flow: after login, go to dashboard if user has multiple
tenants, or direct to first salon if only one

Quality gates:
- Unread message aggregation tested and matches individual salon counts
```

```
- Context switch preserves user session without re-authenticating
- Deep links work (e.g., direct link to "Salon B's messages" from push
notification)
- Empty state and error states properly handled

### Week 6: Hardened Backend Automations
Objectives:
- Cloud Functions for reminders and transitions
- Event-driven notifications pipeline

Deliverables:
- Reminder scheduler
- Booking update triggers (email/push)
- Message-trigger automation baseline

Quality gates:
- Retry logic and idempotency checks in place

### Week 7: Messaging and Waitlist
Objectives:
- In-app messaging and unread state
- Waitlist with slot-open notifications

Deliverables:
- messages and waitlist modules
- Admin bulk messaging v1
- Waitlist auto-notify on opened capacity

Quality gates:
- No tenant cross-talk in messaging payloads

### Week 8: Loyalty and Referrals
Objectives:
- Loyalty ledger + tiering
- Referral mechanics tenant-scoped

Deliverables:
- loyalty fields in customers
- transactions ledger collection
- Admin and client loyalty views

Quality gates:
- Points calculations deterministic and test-covered

### Week 9: Reviews Module
Objectives:
- Review capture from completed bookings
- Moderation and publication controls

Deliverables:
- reviews collection
- Staff/location rating aggregates
- Basic anti-spam checks
```

```
Quality gates:
- Review write only from eligible customers/bookings

### Week 10: Campaigns and Segments
Objectives:
- Marketing segmentation and campaign scheduling
- Activity/challenge engine v1

Deliverables:
- campaigns collection + job runner
- at-risk and inactive segments
- activities/challenges MVP

Quality gates:
- Consent enforcement for outbound channels

### Week 11: Analytics and Reports
Objectives:
- Tenant reports: retention, rebooking, at-risk, campaign performance
- Location/staff performance dashboards

Deliverables:
- Reporting screens + query adapters
- Export endpoints (CSV/JSON)

Quality gates:
- Report numbers match test fixtures and sample calculations

### Week 12: Pilot Launch and Hardening
Objectives:
- Zara as Tenant 1 pilot migration
- Production readiness and playbooks

Deliverables:
- Data migration scripts for Zara bootstrap
- Runbooks: incident, backup, rollback
- Launch checklist and post-launch KPI dashboard

Quality gates:
- Pilot acceptance signoff
- No P0/P1 open defects

### Week 13: Stripe Foundation (Billing + Connect)
Objectives:
- Integrate Stripe Billing for SaaS subscription charging
- Integrate Stripe Connect for salon-side client payments and payouts
- Establish webhook reliability and idempotency

Deliverables:
- Billing customer/subscription lifecycle (monthly + annual)
- Connect onboarding state tracking and account health checks
- Webhooks for invoice/subscription/payment intent/account updates
- Payment audit table and event replay safeguards
```

```
Quality gates:
- No duplicate billing events
- All webhook handlers idempotent and tested
- Secrets management and PCI-safe logging confirmed

### Week 14: Free Trial and Subscription Gating
Objectives:
- Implement configurable free trial lifecycle
- Enforce subscription/trial states on privileged features
- Add grace-period and suspension behavior

Deliverables:
- Trial states: not_started, active, expiring_soon, expired, upgraded
- Trial activation trigger tied to onboarding completion + launch trigger
- Feature gate middleware by tier + trial status
- Trial countdown and upgrade flows in tenant UI

Quality gates:
- Trial expiry jobs are idempotent
- Booking and marketplace access obey subscription state rules
- Reactivation flow tested end-to-end

### Week 15: Salon Onboarding Wizard v1
Objectives:
- Deliver complete salon onboarding from account creation to launch readiness
- Support resumable, mobile-first steps

Deliverables:
- Step flow: Account, Business Profile, Payment Setup, Services, Staff, Policies,
Availability, Marketplace Visibility, Verification & Launch
- Onboarding completion score + blockers list
- Draft-save and resume capability

Quality gates:
- Median onboarding completion time tracked
- Mandatory launch blockers correctly enforced
- All onboarding transitions tested

### Week 16: Client Onboarding Integration v1
Objectives:
- Merge client onboarding with booking/discovery actions
- Keep low-friction guest flow while enabling account upgrade

Deliverables:
- Unified onboarding entry points (booking, save, follow, message)
- Guest-to-full account upgrade path
- Payment method and notification preference setup hooks

Quality gates:
- Conversion from guest to full account measured
- No data loss when upgrading account type

### Week 17: Marketplace Core Launch
Objectives:
```

```
- Ship production marketplace browsing and salon profiles
- Enforce anti-client-theft product rules

Deliverables:
- Feed mode, search mode, profile pages, post view
- Marketplace visibility controls per salon
- "Book this look" deep-linking into booking flow
- No competitor suggestions in active salon booking context

Quality gates:
- Marketplace-to-booking funnel tracked
- Anti-competitor-injection rules validated in UX and API

### Week 18: Marketplace Analytics and Growth Controls
Objectives:
- Provide salon insight into marketplace performance
- Add governance around discovery quality and moderation

Deliverables:
- Metrics: profile views, post engagement, booking conversions, revenue sourced
- Post performance analytics and style-tag reporting
- Abuse detection and content moderation controls

Quality gates:
- Attribution metrics consistent with booking data
- Moderation actions audited and reversible

### Week 19: AI Assistance Release v1
Objectives:
- Release AI features with highest immediate value
- Keep outputs explainable and controllable by salon

Deliverables:
- AI chat assistance
- AI scheduling suggestions
- AI retention risk suggestions
- AI marketing assistant and content generation tools

Quality gates:
- Human override available for all AI actions
- Consent and quiet-hour rules enforced in AI messaging flows

### Week 20: AI Risk Models and Marketplace Personalization
Objectives:
- Add no-show/fraud prediction and discovery personalization
- Harden safety, explainability, and observability

Deliverables:
- No-show/fraud risk scoring with reasons
- Marketplace personalization model using behavior/preferences
- AI observability dashboard (quality, error, fallback rate)

Quality gates:
- Model decisions traceable and reviewable
```

```
- No sensitive attribute inference in production models

---

## Summary Timeline
- **Weeks 1-12**: Core multi-tenant platform and pilot hardening
- **Weeks 13-16**: Payments, free trial, salon onboarding, client onboarding
integration
- **Weeks 17-20**: Marketplace rollout and AI feature layers

## Prompt Engineering Protocol by Phase
For each feature, use this prompt packet structure:
1. Context and business goal
2. Data model and API contract
3. Acceptance criteria (Given/When/Then)
4. Security and tenant-isolation constraints
5. Test cases and edge conditions
6. Required docs to update

This keeps Copilot output consistent and auditable.

---

## 3) Migration Map: Current Zara Modules -> New Multi-Tenant Modules

## 3.1 High-Value Code Reuse Matrix

### Authentication and Profile
Current:
- Login/register/profile flows and validations
- Email change and reset patterns

Target:
- auth module + customer profile module

Action:
- Reuse UI and validation logic
- Rebuild data access layer to tenant-aware repository pattern

Source references:
- src/screens/LoginScreen.js
- src/screens/RegisterScreen.js
- src/screens/ProfileScreen.js
- src/context/AppContext.js

### Booking and Scheduling
Current:
- Booking creation with secure cloud function validation
- Reschedule and status workflows
- Admin schedule operations including swap

Target:
- booking domain service + schedule engine + admin operations
```

```
Action:
- Port secure conflict-prevention function design
- Re-implement with locationId/staffId required fields

Source references:
- src/screens/BookingScreen.js
- src/screens/AppointmentsScreen.js
- src/screens/admin/AdminBookingsScreen.js
- src/screens/admin/AdminScheduleScreen.js
- functions/index.js (createBookingSecure, approveRescheduleSecure)

### Waiting List
Current:
- Date-range waitlist and auto notification logic

Target:
- waitlist module tenant/location-scoped

Action:
- Port matching and auto-remove mechanics
- Add optional staff/service targeting

Source references:
- src/screens/WaitingListScreen.js
- src/context/AppContext.js
- functions/index.js

### Messaging
Current:
- Admin-client messaging, attachments, bulk messaging
- Triggered email/push notifications

Target:
- messaging module + campaign bridge

Action:
- Reuse UI patterns and attachment handling
- Enforce tenant-scoped message partitioning

Source references:
- src/screens/MessagesScreen.js
- src/screens/admin/AdminMessagesScreen.js
- src/context/AppContext.js
- functions/index.js (onMessageCreated)

### Loyalty and Referrals
Current:
- Points, tiers, rewards, referral processing, social-share rewards

Target:
- loyalty domain + referral domain + reward catalog

Action:
- Port transaction-ledger approach
```

```
- Normalize point rules as tenant-level config

Source references:
- src/screens/ClientLoyaltyScreen.js
- src/screens/admin/AdminLoyaltyProgramScreen.js
- src/context/AppContext.js

### Notifications and Automations
Current:
- Scheduled appointment reminders and birthdays
- Booking lifecycle notifications

Target:
- automation service

Action:
- Port scheduler and trigger architecture
- Add tenant-level template and sender branding

Source references:
- functions/index.js

### Reports and Retention
Current:
- Retention/rebooking/at-risk logic

Target:
- analytics module tenant/location/staff scoped

Action:
- Port formulas and risk bins
- Extend to campaign and challenge KPIs

Source references:
- src/screens/admin/AdminReportsScreen.js
- Documentation/05-Features-Implementation/RETENTION_REPORTS_DOCUMENTATION.md

### Gallery and Sharings
Current:
- Admin gallery and client sharing workflows

Target:
- media module

Action:
- Reuse workflows, add tenant moderation policy and storage partition

Source references:
- src/screens/InstagramGalleryScreen.js
- src/screens/ShareNailsScreen.js
- src/screens/admin/AdminGalleryScreen.js
- src/screens/admin/AdminSharingsScreen.js

## 3.2 Migration Sequencing (Practical)
```

```
1. Migrate auth/profile shell
2. Migrate booking + scheduling core
3. Migrate messaging + notifications
4. Migrate waiting list
5. Migrate loyalty/referrals
6. Migrate reports/analytics
7. Migrate gallery/sharings and engagement modules

## 3.3 Data Migration Guidance for Zara as Tenant 1
- Create tenant record for Zara
- Create primary location record
- Map current users to customers or tenantUsers roles
- Backfill bookings with tenantId/locationId and infer staffId placeholder
- Backfill loyalty state and transaction history
- Migrate messages and media with tenant partitioning
- Validate counts before and after migration with checksum reports

---

## 4) VS Code Copilot Delivery Playbook

## Branching Strategy
- trunk: protected, release-ready
- feature/<domain>-<ticket>
- docs/<topic>

## Prompt Templates (Recommended)

### Build Prompt Template
"Implement <feature> in <module>. Respect tenant isolation by requiring tenantId
in all read/write paths. Update Firestore rules and indexes. Add unit tests for
<cases>. Add integration test for <flow>. Update docs in /Documentation/new-
platform/."

### Refactor Prompt Template
"Refactor <file/module> to repository + service layering. Preserve behavior. Add
typed interfaces for inputs/outputs. No UI behavior changes. Ensure all queries
are tenant-scoped and covered by indexes."

### Review Prompt Template
"Review this PR for: tenant data leakage, auth bypass, status-transition bugs,
race conditions in booking, missing tests, and backward-compatibility risks.
Return findings ordered by severity."

## QA Strategy in VS Code
- Mandatory pre-merge checklist in PR description
- Snapshot of affected Firestore indexes per PR
- Security rule diff review mandatory for data-layer PRs
- Nightly regression on booking + notifications + messaging flows

---

## 5) Acceptance Metrics for First Production Window
- Booking success rate >= 98%
```

```
- Double-booking incidents = 0
- Notification delivery success >= 95%
- Median schedule load time < 2.0s
- Tenant data isolation incidents = 0
- Campaign opt-out compliance = 100%

---

## 6) Immediate Next Prompts You Can Run
1. "Create the new repo folder structure and scaffolding for
tenant/location/staff/booking domains with placeholder services and test files."
2. "Implement Firestore schema v1 creation scripts and indexes from this
blueprint."
3. "Generate Firestore security rules enforcing tenant scoping with role checks
for tenant_owner, tenant_admin, location_manager, technician, client."
4. "Implement Week 1 and Week 2 milestones only, with docs and tests."

---

## Appendix A: Suggested Folder Architecture
```text
src/
  app/
    providers/
    navigation/
  domains/
    auth/
    tenants/
    locations/
    staff/
    services/
    bookings/
    waitlist/
    messages/
    loyalty/
    campaigns/
    reviews/
    activities/
    analytics/
  shared/
    ui/
    utils/
    hooks/
    config/
functions/
  bookings/
  notifications/
  campaigns/
  loyalty/
  shared/
docs/
  architecture/
```
```

```
    runbooks/
    api/
```

Appendix B: Naming Standards

- IDs: tenantId, locationId, staffId, customerId, bookingId
- Timestamps: createdAt, updatedAt (serverTimestamp)
- Status fields: status only, with finite enum
- No collection without tenantId unless explicitly global